Measuring the Cost of State: A Controlled Benchmark of Seven React State Management Libraries in Table-Based Dashboards

Ihor Kvasha
Independent Researcher
New York, NY, USA
oneclickaway@kvashaihor.com

***Abstract*—We present a controlled, reproducible benchmark comparing seven React state management approaches—Redux Toolkit, Zustand, Jotai, React Context, Valtio, MobX, and an idiomatic Redux variant—across thirteen interaction scenarios on production builds (React 19.1, Next.js 15.5, TypeScript). The primary finding: selector-based libraries (Redux, Zustand, Jotai, Valtio, MobX), combined with `React.memo`, produce *zero* surplus re-renders when unrelated state changes; React Context produces one re-render per update regardless of which slice changed (RQ1: 50 surplus renders; $1\,632$ ms scenario-level overhead on Apple M5 Max). Three multi-panel scenarios (RQ11–RQ13) confirm isolation generalises beyond a single component tree: all six selector-based libraries confine re-renders to the panel whose slice changed; Context broadcasts to all three panels on every mutation ($3\times$ surplus). Bundle deltas above a 102 KB shared runtime: Redux $+10$ KB, Jotai $+5$ KB, Zustand $+2$ KB; Context adds zero external overhead. Byte-for-byte identical component trees across seven builds, enforced by build-time webpack aliases, isolate subscription model as the sole independent variable; Playwright `toBe` assertions enforce exact render counts. Cross-hardware replication on four platforms and three JS engines confirms hardware- and engine-independence. An unexpected secondary finding: the React Compiler silently over-memoises proxy-based libraries (Valtio, MobX), suppressing required re-renders—an incompatibility invisible to production monitoring; for compiler-compatible conditions the compiler fully eliminates all surplus re-renders, erasing the Context/selector runtime distinction.**

***Index Terms*—React, state management, Redux, Zustand, Jotai, Context API, performance benchmarking, re-render count, bundle size**

I. Introduction

React's ecosystem includes several widely-adopted state management approaches: Redux Toolkit [1] (flux architecture), Zustand [2] (hooks-first store), Jotai [3] (atomic model), and the built-in Context API [4]. Proxy-based and observable libraries such as Valtio [16] and MobX [17] extend the landscape further. Library READMEs, blog posts, and conference talks all advocate for one approach over another. Actual measurements are mostly absent. This study addresses a straightforward question: do these library choices produce measurable differences in runtime behaviour under realistic workloads?

The advice is difficult to verify in practice. A comparison blog post rarely specifies library versions, the component tree topology, whether tree-shaking was enabled, or how interactions were triggered. Without that information, conclusions cannot be reproduced.

A deeper gap exists between what documentation specifies and what happens at runtime. React's documentation specifies that `useContext` re-renders all consumers on any context value change, and that `React.memo` provides a shallow-equality bail-out. What documentation cannot tell you is the *cost* of these behaviours relative to concrete library alternatives, or that field-level subscriptions *and* `React.memo` both need to be in place for complete render isolation—either condition alone is not enough. Documentation describes what a system does; measurement shows what it costs. This paper provides that measurement.

This paper makes three contributions:

1) **A controlled, reproducible benchmark artifact**: the primary scientific contribution. Reproducibility is enforced via a deterministic 100-row dataset; build-time library isolation via a webpack alias enabling genuine tree-shaking (Appendix D); a library-agnostic `StoreAdapter` interface making all JSX byte-for-byte identical across builds; and automated Playwright scripting with exact `toBe(n)` assertions. All results reproduce with a single command (`pnpm benchmark`) in under two minutes on an Apple M5 Max (Section III-F).
2) **Quantitative measurements** of bundle overhead and component re-render counts across thirteen interaction scenarios: RQ1–RQ5 (two-level tree), RQ6 (three-level tree), RQ7 (partial row patches), RQ8 (K-consumer fan-out, $K{=}1, 3, 10$), RQ9 (five-level deep tree), RQ10 (Jotai atom-family per-row isolation), and RQ11–RQ13 (three-panel multi-slice topology with disjoint per-panel subscriptions). All measurements use production builds without profiler overhead.
3) **A decision framework** mapping library choice to application usage profile, with explicit separation between claims backed by measurement and guidance based on community convention.

Section II reviews prior work. Section III describes benchmark methodology. Section IV presents results. Section V discusses implications and limitations. Section VI concludes.

II. Background and Related Work

A. React's Rendering Model

React re-renders a component when (a) its local state changes via `useState` or `useReducer`, (b) a context it subscribes to changes value, or (c) its parent re-renders and the component is not wrapped in `React.memo`. The `React.memo` higher-order component implements a shallow-equality bail-out: if no props have changed since the previous render, the component is skipped during the parent's reconciliation pass.

React 18 introduced automatic batching [5], which groups multiple synchronous state updates from a single event handler into one re-render; test interactions therefore include explicit `await` pauses to keep each operation in its own React update cycle (Section III-D).

The four libraries expose state through different subscription primitives. Redux's `useSelector` runs a selector function and re-renders only when the selected value changes (`===` comparison). Zustand's `useStore` follows the same field-level pattern. Jotai subscribes at the atom level. React Context's `useContext(Ctx)` offers no subscription scoping: any change to the context value object re-renders all consumers regardless of which properties they read.

B. Prior Work

Krausest's *js-framework-benchmark* [6] measures DOM-manipulation throughput across dozens of frameworks including React, operating at the framework-selection level. It does not address state management library choice within a single framework.

Ollila et al. [11] extend the analysis to five declarative frameworks, finding that minimising reconciliation input size yields orders-of-magnitude performance advantages under large tree updates. This motivates locating the performance variable at the subscription level within React. Diniz-Junior et al. [12] measure Angular, React, and Vue across build size, time-to-interactive, and DOM-manipulation time; neither study addresses state management library choice within React.

Selakovic and Pradel [9] catalogue 50 recurring JavaScript performance-optimisation patterns in real-world applications; their findings motivate measuring bundle size alongside render counts.

The most closely related peer-reviewed work is Pronina and Kyrychenko [8], who compare Redux Toolkit with React hooks in a CRUD-style SPA, measuring page load time and heap memory. Render counts at the component level are not measured, no memoisation strategy is described, and the study predates Zustand v5 and Jotai v2.

Valtio [16] (v2.3, proxy-based) and MobX [17] (v6.15, observable-based) notify components only when their accessed fields change, a mechanism equivalent to field-level selectors under Redux, Zustand, and Jotai. This study extends the primary four-library comparison with confirmatory replication runs for both libraries: Valtio and MobX produced render counts byte-identical to the selector-based libraries across all tested scenarios (12/12 Playwright assertions pass for each; Table V). MobX carries a +17 KB bundle overhead—the largest of the tested libraries—while Valtio's +3 KB falls between Zustand (+2 KB) and Jotai (+5 KB) (Table II). TanStack Store remains outside the study scope.

A systematic literature search of IEEE Xplore, ACM Digital Library, Scopus, Springer Link, and ScienceDirect was conducted in April 2026 (no publication-date restriction) using queries `"React" AND "state management" AND "benchmark"` and `"React" AND "re-render" AND "performance"`. Inclusion criteria: (i) peer-reviewed venue, (ii) React as the primary framework, (iii) quantitative performance metric compared across two or more React state management libraries under a controlled component topology. No record satisfied all three criteria simultaneously. The contribution of this work is therefore not a claim about the absence of undiscovered prior work, but the controlled-topology artifact design that makes the result *interpretable*: the `StoreAdapter` interface, byte-for-byte identical JSX across builds, and exact `toBe(n)` Playwright assertions are the mechanisms through which subscription model is isolated as the sole independent variable — a design constraint that criterion (iii) was constructed to capture. The five-database search confirms that this combination has not appeared in peer-reviewed, archivally indexed form. Community benchmarks by practitioners and library authors informed library selection and confirmed the gap, but fall outside the scope of a peer-reviewed literature search. Studies measuring render behaviour under less controlled conditions, or using broader performance proxies, may exist in venues not covered by these five databases. Criterion (iii) is a validity constraint, not a way to exclude prior work by definition: a controlled topology is necessary to isolate the subscription model as the only independent variable.

III. Methodology

A. Benchmark Application Design

The benchmark is a single-page dashboard implemented in Next.js 15.5 (App Router, statically pre-rendered, served via `next start`) with React 19.1 and TypeScript. Two measured components exist: `Dashboard`, which owns a live counter and hosts the data table, and `DataTable`, which renders the 100-row table with filter, sort, and per-row selection controls. Fig. 1 shows the two component tree topologies.

Scope note: The benchmark intentionally uses a minimal two-component tree to isolate the subscription model as the sole independent variable; results should be read within this controlled topology and may not transfer directly to production dashboards with many independent subscribing components. **React Compiler scope boundary:** All measurements use React 19.1 with the React Compiler [13] disabled, which remains the default configuration as of April 2026. The compiler's effect on all seven conditions—including proxy incompatibility with Valtio and MobX—is evaluated in Appendix H.

The dataset is generated deterministically using a large-prime modulo (`(i * 7919) % 1000`), with no `Math.random()`, so

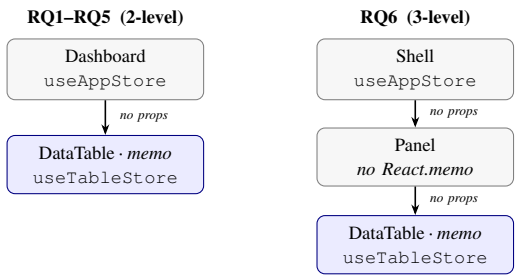

Fig. 1. Component tree topologies. Arrows denote parent–child relationships; no props cross any edge. Blue-shaded nodes carry a `React.memo` boundary.

the same 100 rows are produced on every build. The global state shape is defined by a shared `StoreAdapter` interface:

```
export interface StoreAdapter {
  state: AppState    // rows, filter, sortBy, sortDir,
                     // counter, selectedIds
  actions: AppActions
}
```

All four library implementations satisfy this interface. The component tree references only `StoreAdapter`, making all JSX byte-for-byte identical across builds; any observable difference is attributable solely to the state layer. The 100-row dataset falls below the threshold at which virtual scrolling is typically introduced, preventing visualization behaviour from conflating re-render counts. The 100-row count was selected to represent a realistic dense-table workload; commonly used virtualisation libraries (e.g., TanStack Virtual, React Window) are applied at substantially larger row counts, typically hundreds to thousands of rows. The impact of row count on reconciliation cost is not independently varied in this study; results should be interpreted within the non-virtualised dense-render regime.

B. Library Isolation

Each library is an independent entry point under `store/`. The active entry is selected at build time via a webpack module alias in `next.config.mjs`:

```
config.resolve.alias[
  path.resolve('./store/active.tsx')
] = path.resolve(
  `./store/${process.env.NEXT_PUBLIC_STATE_LIBRARY}/index.
    tsx`
)
```

The alias key is the **absolute filesystem path** of `store/active.tsx` (see Appendix D for the rationale). This enables genuine tree-shaking: only the selected library's code is included in the output. Each benchmark run begins with `rm -rf .next` to prevent stale chunk reuse.

C. Render Tracking

`Dashboard` and `DataTable` are instrumented with a `<RenderTracker id>` wrapper whose `useEffect` (no dependency array) fires after every render commit and pushes a record to `window.__renderRecords`. Playwright reads this array via `page.evaluate()` after each scenario.

React's `<Profiler>` API [14] was evaluated as an alternative. In standard `react-dom` production builds the `onRender` callback is a no-op; `react-dom/profiler` adds instrumentation overhead that would perturb the measured artifact. The `useEffect` approach avoids this. The trade-off is that render *duration* (`actualDuration`) is unavailable; render *count* is therefore the primary dependent variable. A secondary *scenario elapsed time* metric (termed render-span for brevity) approximates per-scenario wall-clock cost as the elapsed time between the first and last `performance.now()` timestamps in `window.__renderRecords` for a given component. **Interpretation note:** this metric encompasses React reconciliation, browser paint, Playwright input dispatch, and inter-event idle time; it is *not* an isolated measure of reconciliation CPU cost (React 18 automatic batching [5] requires explicit `await` gaps between interactions, and these idle gaps are included in scenario elapsed time). Table VI reports mean [95% CI] over N=20 runs (run 1 excluded as warm-up; N=19 steady-state observations; t-distribution, df=18 [10]). The `performance.now()` timestamps are recorded inside `useEffect` callbacks (after each browser paint) and are React-internal, not Playwright-internal. In RQ1, Context's 50 `useEffect` invocations capture the full wall-clock span including all inter-click idle time, so render-span measures scenario elapsed time rather than isolated reconciliation CPU cost.

D. Subscription Scoping Design

Global state has two semantically independent regions: *table state* (`rows`, `filter`, `sortBy`, `sortDir`, `selectedIds`) and *counter state* (`counter`). Each library exposes two public hooks. `useAppStore()` subscribes to all of `AppState` and is used by `Dashboard`. `useTableStore()` subscribes to table state only, excluding `counter`, and is used by `DataTable`, which is also wrapped in `React.memo`:

```
export const DataTable = memo(function DataTable() {
  const { rows, filter, sortBy, sortDir,
          selectedIds, toggleSelect } = useTableStore()
  // ...
})
```

Both conditions are required for render isolation under RQ1. `useTableStore()` prevents the library's subscription from firing when only `counter` changes. `React.memo` prevents React's reconciliation cascade when `Dashboard` (the parent, subscribed to `counter`) re-renders. Either condition alone is insufficient.[1]

For Context, `useTableStore()` still reads `useContext(StateCtx)`, which fires on *any* context value change regardless of which property changed, the architectural property under test in RQ1. A dual-context

[1]Redux's `useAppStore()` uses a single `useSelector((s) => s.app)` call. Because RTK's `immer` produces a new object reference on every dispatch, `Dashboard` re-renders on all five scenario types equally. This asymmetry does not affect `DataTable` counts (Table III), which use `useTableStore()` with consistent per-field resolution. The coarser selector is preserved intentionally as a common pre-optimisation adoption pattern. Note that this pattern does *not* conform to the Redux Style Guide [7], which recommends per-field selectors; `Dashboard`-level render counts therefore measure Redux under a common footgun rather than its idiomatic best case. A per-field `useAppStore()` variant is identified as a fifth condition in this study (`redux-idiomatic`; results in Table III).

TABLE I
INTERACTION SCENARIOS AND RESEARCH QUESTIONS

Scenario	Interaction	Ops	Metric
RQ1	Counter increment	50	`DataTable` re-renders
RQ2	Filter input (keystrokes)	5	`DataTable` re-renders
RQ3	Row selection toggle	20	`DataTable` re-renders
RQ4	Sort direction toggle	10	`DataTable` re-renders
RQ5	Row dataset refresh	5	`DataTable` re-renders
RQ6	Counter increment (3-level tree)	50	All 3 components
RQ7	Single-field row patch	10	Dashboard + DataTable
RQ8	K-consumer fan-out (K=1, 3, 10)	50	Total surplus renders
RQ9	Counter incr. (5-level deep tree)	50	All 5 components
RQ10	Single-field row patch (atom-family)	10	Per-row re-renders
RQ11	Counter incr. (3-panel multi-slice)	50	Per-panel re-renders
RQ12	Filter input (3-panel multi-slice)	5	Per-panel re-renders
RQ13	Row selection (3-panel multi-slice)	20	Per-panel re-renders

split (`StateCtx` and `DispatchCtx`) is employed, a common optimisation that does not change the RQ1 outcome.

Zustand v5 selector pitfall (important for replication): Zustand v5 uses `Object.is` for selector equality. Inline object-literal selectors (e.g., `useStore(s => ({a: s.a}))`) always produce new references and cause infinite re-render loops; this is a silent runtime defect, not a compile-time error. Per-field primitive selectors are safe; object-returning selectors require `useShallow`. This constraint was applied throughout the benchmark implementation.

E. Interaction Scenarios

Thirteen research questions (RQ1–RQ13) are covered (Table I). RQ1–RQ5 operate on the primary two-level tree; RQ6 uses a dedicated three-level route (`Shell` → `Panel` → `DataTable`) with an unmemoised intermediate component to test whether leaf isolation holds across a deeper tree; RQ7 exercises partial single-field row patches; RQ8 measures K-consumer fan-out at K=1, K=3, and K=10 (three sub-conditions). RQ9 extends the depth study with three additional unmemoised intermediate layers (five-level tree). RQ10 tests Jotai atom-family per-row isolation outside the uniform `StoreAdapter` interface. Three further scenarios (RQ11–RQ13) test a *multi-panel topology*: three independently memoised panels each subscribe to a disjoint state slice (PanelA → counter; PanelB → filter/sort/rows; PanelC → selection), exercising whether selector-based libraries confine re-renders to the panel whose slice actually changed.

All interactions are sequentialised with `await` to prevent React 18 automatic batching. In RQ2, `pressSequentially('Alpha, {delay: 50})` ensures each keystroke is committed as a separate React update cycle before the next is dispatched. Assertions are exact (`toBe(n)`): because all interactions are deterministically scripted, render counts are invariant by construction, and exact assertions enforce this invariance.

Scenarios execute in fixed order RQ1–RQ7 for the render-count suite (render counts are deterministic and exact regardless of trial order; assertions enforce this invariance via `toBe(n)`). For the render-span timing suite, the five RQ1–RQ5 scenario bodies are executed in a *per-run randomised order* within a single combined Playwright test using a seeded deterministic shuffle (linear-congruential PRNG, seed = run index). All five code paths are thus executed in each of the N=20 runs before steady-state observations are drawn, providing a single cross-scenario JIT warm-up run (run 0) after which V8 has JIT-compiled all paths. Runs 1–19 (N=19 steady-state observations) each present the five scenarios in a distinct, deterministic order, distributing any residual V8 tier-up or CPU frequency scaling effects across all scenarios rather than confounding position-dependent scenarios.

F. Bundle Size Measurement

Bundle size is measured as First Load JS delta above the 102 KB shared baseline, as reported by `next build`. All primary comparisons use **uncompressed** First Load JS; gzip totals of `.next/static/chunks` are a secondary cross-check. Bundle sizes were verified stable across N=3 independent clean builds per library (byte-identical output, consistent with [10]). Timing runs use N=20 (run 1 excluded as warm-up; N=19 steady-state observations) to support formal 95% confidence interval construction.

The test environment was a MacBook Pro 14-inch (Apple M5 Max, 128 GB unified memory) running macOS 26.4 Tahoe, Node.js v22.14.0, and pnpm v9.15.4. Primary benchmark runs used Playwright 1.59.1 with Chromium (V8); the Firefox cross-validation run used Firefox 148.0.2 (SpiderMonkey) via Playwright's *Desktop Firefox* device profile (Appendix E). **Hardware note: Absolute render-span values scale with hardware speed.** The Apple M5 Max represents an unusually high-performance configuration; absolute values in Table VI are expected to be substantially longer on mid-range x86-class hardware (developer laptops, CI runners, cloud VMs). Absolute render-span results should therefore be treated as lower bounds on timing for comparable hardware; relative comparisons *within a single hardware platform* are unaffected by machine speed. The render-*count* results (Tables III–VII) are hardware-independent. Cross-hardware replication on three additional platforms (an Apple M3 Max 128 GB MacBook Pro, independent collaborator machine; an Intel Core i9 MacBook Pro; and an Apple M1 GitHub Actions runner) is reported in Appendix G.

IV. RESULTS

A. Bundle Overhead

Table II reports uncompressed First Load JS and the delta above the 102 KB shared baseline. All three independent clean builds per library produced byte-identical figures, confirming webpack output determinism. Fig. 2 visualises the deltas.

Redux's +10 KB overhead reflects RTK's action-creator utilities, reducer helpers, and `immer`. Although RTK bundles `reselect`, the benchmark does not call `createSelector`, so `reselect` contributes zero to the delta. Redux's per-route chunk (2.10 KB) is *smaller* than Zustand's (2.49 KB), yet its First Load JS total is 8 KB larger: RTK's dependencies land in shared chunks loaded on every navigation, inflating First Load

TABLE II
BUNDLE OVERHEAD PER LIBRARY. PRIMARY: FIRST LOAD JS DELTA ABOVE 102 KB SHARED BASELINE. REACT CONTEXT HAS **ZERO** EXTERNAL LIBRARY OVERHEAD; ITS 104 KB TOTAL IS INDISTINGUISHABLE FROM THE BASELINE AT 1-KB REPORTING PRECISION; THE NOMINAL +2 KB REFLECTS BENCHMARK ADAPTER CODE PRESENT IN ALL CONFIGURATIONS, NOT AN NPM DEPENDENCY.†

Library	First Load	Delta	Gzip
Redux Toolkit v2.5 (coarse)	112 KB	**+10 KB**	244.5 KB
Redux Toolkit v2.5 (idiomatic)	112 KB	**+10 KB**	244.7 KB
Jotai v2.12	107 KB	**+5 KB**	240.0 KB
Zustand v5.0	104 KB	**+2 KB**	238.0 KB
React Context (built-in)	104 KB	**0 KB**†	237.2 KB
Valtio v2.3 (proxy)	105 KB	**+3 KB**	245.9 KB
MobX v6.15 (observable)	119 KB	**+17 KB**	260.1 KB

†Context is a built-in React primitive with no npm dependency. The 2 KB above the 102 KB baseline is benchmark adapter code common to all configurations.

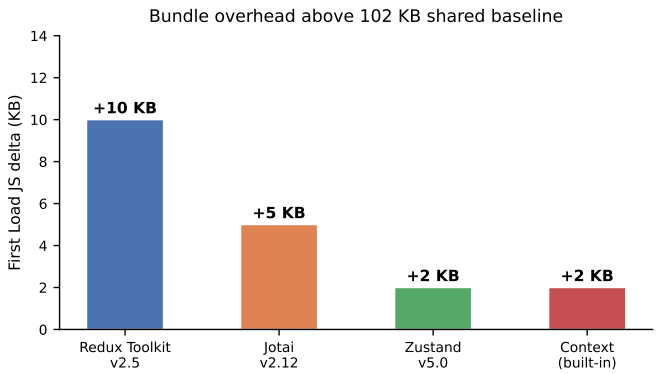

Fig. 2. Bundle overhead delta per library (uncompressed First Load JS). Lower is better. Context's bar reflects benchmark adapter code common to all configurations; the library itself adds 0 KB (see Table II).

JS without appearing in the per-route chunk. This illustrates that per-route chunk size is a misleading bundle metric when dependency graphs are hoisted into shared chunks; First Load JS, which subsumes all shared-chunk contributions, is the more accurate measure of delivery cost. A symmetric shared-chunk breakdown for Jotai and Zustand was not conducted; the reported First Load JS figures capture each library's full contribution but do not separate per-route from shared-chunk portions. The two Redux variants share a nearly byte-identical bundle (+10 KB each); the second variant adds only per-field `useSelector` calls and contributes no additional dependency weight. **Context bundle cost.** React Context is a built-in React primitive carrying zero external library overhead. Its 104 KB First Load JS equals the Zustand total in absolute terms; however, Zustand's +2 KB delta represents a genuine npm dependency whereas Context's same nominal delta is entirely benchmark adapter code common to all configurations. In bundle terms, Context is effectively free. Gzip totals (Table II) broadly track uncompressed deltas: MobX (260.1 KB) $>$ Valtio (245.9 KB) $>$ Redux (244.5 KB) $>$ Jotai (240.0 KB) $>$ Zustand (238.0 KB) $\approx$ Context (237.2 KB), confirming consistency across measurement methods. Note that Valtio's gzip rank (above Redux) exceeds its uncompressed-delta rank (+3 KB vs. +10 KB): proxy-polyfill code compresses less efficiently than Redux Toolkit's action-creator utilities. **Valtio and MobX bundle cost.** Valtio (v2.3) adds +3 KB above the 102 KB shared baseline, ranking between Zustand (+2 KB) and Jotai (+5 KB) by uncompressed delta. MobX (v6.15) adds +17 KB—the largest overhead of the tested libraries—reflecting `mobx`'s reactive data engine and `mobx-react-lite`'s observer context.

TABLE III
DATATABLE RE-RENDER COUNTS PER SCENARIO AND LIBRARY. VALTIO (V2.3) AND MOBX (V6.15) REPLICATIONS CONFIRMED IDENTICAL; SEE TABLE V.

Scenario	Redux	Rdx-Idm	Jotai	Zustand	Context
RQ1 — counter ×50	**0**	**0**	**0**	**0**	**50**
RQ2 — filter ×5	5	5	5	5	5
RQ3 — select ×20	20	20	20	20	20
RQ4 — sort ×10	10	10	10	10	10
RQ5 — rows ×5	5	5	5	5	5
RQ7 — patch ×10	10	10	10	10	10

B. Re-render Counts

Table III reports `DataTable` re-render counts per scenario and library. Table IV reports `Dashboard` counts (identical across all seven tested conditions), validating measurement calibration. **Design note:** Redux's `useAppStore()` uses a deliberately coarse selector, `(s) => s.app`, rather than per-field selectors. This is a common initial adoption pattern before per-field memoisation is introduced, but it does *not* conform to the Redux Style Guide [7], which recommends per-field selectors. **`Dashboard`-level render counts (Table IV) therefore measure Redux under a common pre-optimisation pattern (*not* its idiomatic best case) and should not be used in isolation to assess Redux's render-count performance.** `DataTable` counts (Table III) are unaffected: `useTableStore()` applies consistent per-field resolution across all libraries.[2] Table VI reports inter-library render-span timing.

RQ1 yields the study's primary finding: Context produced 50 surplus re-renders; all six selector-based conditions (Redux, redux-idiomatic, Jotai, Zustand, Valtio, and MobX) produced zero: a complete elimination of wasted work. Each counter increment changes `StateCtx`'s value object, propagating to all `useContext(StateCtx)` consumers regardless of which properties changed. Redux, Zustand, Jotai, Valtio, and MobX each detect that their subscribed fields (`rows`, `filter`, `sortBy`, `sortDir`, `selectedIds`) have not changed and skip the render entirely (see Section III-D). The 1632 ms render-span for

[2]In the tested scenarios all dispatches modify at least one field of `s.app`, so the coarse selector produces identical `Dashboard` counts to per-field subscriptions. A per-field Redux `useAppStore()` variant is included as a fifth condition in this study (`redux-idiomatic`; results in Table III).

TABLE IV
DASHBOARD RE-RENDER COUNTS PER SCENARIO AND LIBRARY (IDENTICAL ACROSS ALL LIBRARIES, VALIDATES MEASUREMENT CALIBRATION)

Scenario	Redux	Rdx-Idm	Jotai	Zustand	Context
RQ1 — counter ×50	50	50	50	50	50
RQ2 — filter ×5	5	5	5	5	5
RQ3 — select ×20	20	20	20	20	20
RQ4 — sort ×10	10	10	10	10	10
RQ5 — rows ×5	5	5	5	5	5
RQ7 — patch ×10	10	10	10	10	10

TABLE V
VALTIO (V2.3) AND MOBX (V6.15) RENDER-COUNT REPLICATION (PRIMARY ROUTE, M5 MAX). BOTH PROXY/OBSERVABLE LIBRARIES PRODUCE COUNTS BYTE-IDENTICAL TO THE SELECTOR-BASED PRIMARY CONDITIONS (TABLE III). RQ6 (DEPTH 3), RQ9 (DEPTH 5), AND RQ8 FAN-OUT CONFIRMED: SHELL/PANEL/LAYERS = 50, DATATABLE = 0, SURPLUS = 0 AT K=3 AND K=10. TESTS: 12/12 EACH. TIMING (N=20, SAME STEADY-STATE PROTOCOL) REPORTED IN TABLE VI.

Scenario	Dashboard		DataTable	
	Valtio	MobX	Valtio	MobX
RQ1 — counter ×50	50	50	**0**	**0**
RQ2 — filter ×5	5	5	5	5
RQ3 — select ×20	20	20	20	20
RQ4 — sort ×10	10	10	10	10
RQ5 — rows ×5	5	5	5	5
RQ7 — patch ×10	10	10	10	10

Context represents the wall-clock time across the full 50-increment sequence (including inter-click idle time) during which `DataTable`'s reconciliation is invoked on all 50 passes that the selector-based libraries avoid. This 1632 ms figure is a scenario-level wall-clock total encompassing Playwright input dispatch, event propagation, commit scheduling, and inter-click idle; it should *not* be read as pure reconciliation overhead (see Section V for contextual scale framing). Because render counts are enforced as exact deterministic assertions (`toBe(`n`)` in Playwright), they carry no sampling uncertainty and require no statistical interval: the count either passes or the test suite fails.

RQ2–RQ5 produce identical counts across all seven tested conditions (5, 20, 10, and 5 respectively). When the changed state *is* consumed by the subscribing component, there is no wasted rendering regardless of library. The performance advantage of selector-based subscription is strictly zero in these scenarios.

RQ6 tests render isolation in a three-level tree where the intermediate component (`Panel`) is deliberately *not* wrapped in `React.memo` (Table VII). `Shell` re-renders once per counter increment for all six selector-based conditions; `Panel` cascades 50 re-renders from the parent, also uniformly. `DataTable`'s `React.memo` bail-out fires for all selector-based conditions (zero renders), while Context produces 50. This confirms that render isolation at a memoised leaf survives intermediate unmemoised layers.

RQ9 extends the depth study of RQ6 by inserting three additional unmemoised intermediate layers (`Layer1`–`Layer3`) between the state-subscribing root (`DeepShell`) and the `React.memo`-guarded leaf (`DataTable`), producing a five-level tree. The five-level route replaces all five selector-based libraries' `DataTable` re-renders with 0, while Context continues to produce 50 (Table VIII). The result extends the RQ6 depth-invariance finding: `React.memo` isolation at a leaf survives arbitrary numbers of unmemoised intermediate layers, regardless of tree depth.

RQ7: ten consecutive single-field row patches (`patchRow('row-0', {value: prev + 1})`) produced 10 `Dashboard` re-renders and 10 `DataTable` re-renders across all seven tested conditions. Although Redux's `immer` preserves unchanged row object references (structural sharing), all seven `useTableStore()` implementations subscribe to the `rows` *array* reference. Every `patchRow()` call produces a new array reference via `.map()`, triggering a re-render in all libraries. The RQ7 equivalence therefore characterises *all seven conditions subject to the array-reference subscription constraint of the uniform `StoreAdapter` interface*, not the libraries' intrinsic capabilities under per-row update workloads.

RQ10 tests the per-row subscription capability that lies *outside* the uniform `StoreAdapter` interface. A dedicated `/atom-family` route implements one Jotai atom per row: `rowAtomMap` stores 100 individual `atom<Row>` instances; each `RowCell` component subscribes exclusively to its own row atom. Applying 10 consecutive patches to `row-0` produced 10 re-renders in `RowCell-row-0` and exactly 0 re-renders in every other `RowCell` (rows 1–99) as well as in the `AtomFamilyTable` wrapper (which reads only the stable `rowIdsAtom`). Compared with RQ7's baseline of 10 `DataTable` re-renders encompassing all 100 rows, the atom-family pattern triggers reconciliation of only 1 of 100 `RowCell` components per patch; the other 99 remain idle, as does the `AtomFamilyTable` wrapper, confirming the theoretical per-row isolation advantage of Jotai's atomic subscription model under single-row update workloads.

C. Multi-Panel Topology (RQ11–RQ13)

RQ11–RQ13 evaluate whether subscription-level isolation generalises beyond the canonical two-component Dashboard+DataTable tree to a *multi-panel topology* in which three independently memoised panels each subscribe to a disjoint state slice: **PanelA** → `useCounterSlice()` (counter only), **PanelB** → `useFilterSortSlice()` (rows, filter, sortBy, sortDir), **PanelC** → `useSelectionSlice()` (selectedIds). Each panel is instrumented with a `RenderTracker` recording every render commit; controls outside the panels drive the store without being tracked.

Table IX shows the render counts across all seven conditions. The result is unambiguous: all six selector-based libraries (Redux, redux-idiomatic, Zustand, Jotai, Valtio, MobX) produce

TABLE VI

SUPPLEMENTARY: DATATABLE SCENARIO ELAPSED TIME (INDICATIVE WALL-CLOCK COST). MEAN [±95% CI] IN MS, N=19 STEADY-STATE RUNS (t-DISTRIBUTION, df=18, RUN 1 EXCLUDED AS WARM-UP [10]). RENDER COUNT (TABLE III) IS THE PRIMARY DEPENDENT VARIABLE; THIS TABLE APPROXIMATES SCENARIO-LEVEL WALL-CLOCK COST AND ENCOMPASSES PLAYWRIGHT OVERHEAD, EVENT PROPAGATION, AND INTER-EVENT IDLE TIME — IT IS NOT AN ISOLATED RECONCILIATION MEASURE. --- INDICATES ZERO RENDERS (RQ1 SELECTOR-BASED LIBRARIES). THE RQ1 CONTEXT VALUE (1632.1 MS) IS HARDWARE-DEPENDENT; SEE TABLE XVIII FOR CROSS-PLATFORM COMPARISONS. SCENARIO ORDER IS RANDOMISED PER RUN (SEEDED DETERMINISTIC SHUFFLE, SEED = RUN INDEX); INTER-LIBRARY COMPARISONS WITHIN EACH SCENARIO ROW ARE VALID; CROSS-SCENARIO COMPARISONS ARE INDICATIVE ONLY (SEE SECTIONS III AND V-D). FOR RQ2–RQ5, SUB-2 MS INTER-LIBRARY DELTAS PRIMARILY CONFIRM TEST HARNESS CONSISTENCY ACROSS LIBRARY RUNS RATHER THAN ISOLATING RECONCILIATION CPU COST. **RDX-IDM** = REDUX-IDIOMATIC (PER-FIELD USESELECTOR AT DASHBOARD LEVEL; SEE SECTION V-D). VALTIO AND MOBX TIMING COLLECTED IN A DEDICATED N=20 PASS (SAME STEADY-STATE PROTOCOL; RUN 1 WARM-UP EXCLUDED).

Scenario	Redux	Rdx-Idm	Jotai	Zustand	Context	Valtio	MobX
RQ1 — counter ×50	—	—	—	—	1632.1 [±0.5]	—	—
RQ2 — filter ×5	213.2 [±0.7]	212.9 [±1.5]	213.7 [±0.7]	212.6 [±1.2]	213.0 [±0.7]	211.7 [±0.9]	212.3 [±0.9]
RQ3 — select ×20	632.0 [±0.6]	631.4 [±0.7]	631.5 [±0.6]	631.7 [±0.6]	631.9 [±0.7]	632.9 [±1.9]	631.4 [±1.0]
RQ4 — sort ×10	297.5 [±0.6]	297.2 [±0.6]	297.5 [±0.4]	298.3 [±2.1]	298.1 [±1.8]	297.9 [±0.6]	297.5 [±1.9]
RQ5 — rows ×5	131.7 [±0.5]	131.9 [±0.6]	131.7 [±0.7]	131.7 [±0.5]	131.1 [±0.5]	130.8 [±0.5]	130.5 [±0.4]

TABLE VII

RQ6 — RE-RENDER COUNTS ACROSS ALL THREE LEVELS OF THE NESTED TREE (50 COUNTER INCREMENTS).

Component	Rdx	RI	Jtai	Zstd	Valt	MobX	Ctx
Shell (root)	50	50	50	50	50	50	50
Panel (no memo)	50	50	50	50	50	50	50
DataTable (leaf)	**0**	**0**	**0**	**0**	**0**	**0**	**50**

Rdx=Redux, RI=redux-idiomatic, Jtai=Jotai, Zstd=Zustand, Valt=Valtio, Ctx=Context.

TABLE VIII

RQ9 — RE-RENDER COUNTS IN THE 5-LEVEL TREE (50 COUNTER INCREMENTS). **0** ENTRIES CONFIRM REACT.MEMO LEAF ISOLATION PERSISTS THROUGH THREE ADDITIONAL UNMEMOISED INTERMEDIATE LAYERS.

Component	Rdx	RI	Jtai	Zstd	Valt	MobX	Ctx
DeepShell (root)	50	50	50	50	50	50	50
Layer1 (no memo)	50	50	50	50	50	50	50
Layer2 (no memo)	50	50	50	50	50	50	50
Layer3 (no memo)	50	50	50	50	50	50	50
DataTable (leaf)	**0**	**0**	**0**	**0**	**0**	**0**	**50**

Rdx=Redux, RI=redux-idiomatic, Jtai=Jotai, Zstd=Zustand, Valt=Valtio, Ctx=Context.

TABLE IX

RQ11–RQ13 — MULTI-PANEL RENDER COUNTS. PA = PANELA (USECOUNTERSLICE), PB = PANELB (USEFILTERSORTSLICE), PC = PANELC (USESELECTIONSLICE). **BOLD** ENTRIES HIGHLIGHT SURPLUS RE-RENDERS.

Library	**RQ11** (50 counter incr.)			**RQ12** (5 filter keys)			**RQ13** (20 sel. toggles)		
	PA	PB	PC	PA	PB	PC	PA	PB	PC
Redux	50	0	0	0	5	0	0	0	20
Rdx-Idm.	50	0	0	0	5	0	0	0	20
Zustand	50	0	0	0	5	0	0	0	20
Jotai	50	0	0	0	5	0	0	0	20
Valtio	50	0	0	0	5	0	0	0	20
MobX	50	0	0	0	5	0	0	0	20
Context	50	**50**	**50**	**5**	5	**5**	**20**	**20**	20

Rdx-Idm. = redux-idiomatic. Bold: surplus renders in panels whose subscribed slice did not change.

zero re-renders in the two panels whose slices were not changed, regardless of which scenario drove the mutation. React Context, with its all-or-nothing subscription to the shared `StateCtx` value, broadcasts to all three panels on every mutation, producing a 3× surplus in all cross-slice scenarios.

The six selector-based libraries achieve zero surplus renders through mechanistically different routes: Redux and redux-idiomatic via per-field `useSelector`; Zustand and Jotai via fine-grained per-field or per-atom selectors; Valtio via per-instance `prevValue` closures gating `useSyncExternalStore` notifications; MobX via `reaction()` trackers that observe only the declared slice fields. The consistent outcome confirms that the multi-panel isolation property is a consequence of the *field-level subscription contract*, not any specific library implementation.

V. DISCUSSION

A. The Render Isolation Tradeoff

The RQ1 result demonstrates that React Context imposes a render cost proportional to the state update frequency of *any* slice in the shared context, not only the slices a component cares about. The RQ6 results confirm that this isolation extends across a three-level tree: `React.memo` at the leaf is sufficient regardless of intermediate tree depth.

One important nuance: `DataTable`'s `useMemo` guard on `visibleRows` prevents the filter-and-sort computation from rerunning during Context's 50 surplus renders, because the Context reducer preserves stable field references when only `counter` changes. The 50 extra reconciliations incur only React reconciliation overhead, not recomputation cost. In real applications where a hot state slice also writes fields consumed by the table (e.g., a WebSocket price feed updating both a counter and row values simultaneously), the `useMemo` guard would not help and the full $O(n \log n)$ recomputation would be incurred on every surplus render.

TABLE X
RQ8 — K-CONSUMER FAN-OUT: TOTAL SURPLUS RE-RENDER COUNTS UNDER 50 COUNTER INCREMENTS. CONTEXT SCALES AS $K\times50$; ALL SIX SELECTOR-BASED CONDITIONS PRODUCE ZERO AT EVERY K.

K	Rdx	RI	Jtai	Zstd	Valt	MobX	Ctx
K=1 (baseline)	0	0	0	0	0	0	**50**
K=3	0	0	0	0	0	0	**150**
K=10	0	0	0	0	0	0	**500**

Rdx=Redux, RI=`redux-idiomatic`, Jtai=Jotai, Zstd=Zustand, Valt=Valtio, Ctx=Context.

Consumer-count fan-out (RQ8). The main benchmark uses a single `DataTable` consumer (K=1). To empirically validate the $K\times N$ scaling claim, I added RQ8: a dedicated `/k-consumer` route renders K identical `React.memo`-wrapped `DataTable` instances, each subscribing to the same store. Table X reports total surplus render counts for K=1, K=3, and K=10 under 50 counter increments. Context's surplus renders scale exactly as $K\times N$: 50 at K=1, 150 at K=3, and 500 at K=10. All three selector-based libraries produce zero surplus renders at every K value tested. Fan-out linear scaling is empirically confirmed: the cost is proportional to the number of consumers, not to the store library's subscription overhead.

Redux, Zustand, Jotai, Valtio, and MobX achieve equivalent render isolation through different mechanisms. The mechanism differs; the outcome (zero unnecessary re-renders when unrelated state changes) is identical when each library is used idiomatically. Render-span timing was not collected for RQ8, as the deterministic count result fully characterises the fan-out behaviour; wall-clock cost scales linearly from the per-render latency reported in Table VI. One practical point worth emphasising: `React.memo` is a *necessary partner* to field-level subscriptions when the parent subscribes to the changing state slice. Without it, parent re-renders cascade to all children regardless of subscription scope. Alternative topology patterns that place consumers *outside* the re-rendering subtree (i.e., no ancestor of the consumer subscribes to the changing slice) can achieve isolation without memoisation; this benchmark uses the parent-child topology because it represents the most common production dashboard pattern.

B. Bundle Size vs. Runtime Render Cost

For applications where state slices update together or infrequently (CRUD, form-heavy, content management), Context's RQ1 overhead is rarely observable, and its zero-overhead bundle profile makes it a strong default choice.

For applications with high-frequency independent state updates (real-time dashboards, multiplayer tools, data feeds), Redux, Zustand, or Jotai provide meaningful runtime savings at the cost of a small bundle increase. Among the three, Zustand's +2 KB overhead makes it the most cost-efficient choice when atomic state (Jotai) or Redux's ecosystem (DevTools, middleware, code generation) is not required.

The avoidable scenario-level overhead incurred by Context in RQ1 is 1632.1 ms (M5 Max, Table VI) versus 0.0 ms for all four selector-based libraries over the same 50-increment burst; the M3 Max figure is 1634.2 ms (Table XVIII). **Interpretation caveat:** these totals encompass Playwright input dispatch, event propagation, commit scheduling, and inter-click idle time. Dividing the scenario total by the render count to derive a per-render reconciliation cost is not meaningful: the denominator (50 React commits) does not correspond to the numerator (scenario elapsed time including idle), so the quotient does not isolate reconciliation CPU cost. At 1.6 seconds of scenario-level overhead for a 50-increment burst (consistent across M5 Max and M3 Max), the surplus is macroscopically user-visible under high-frequency update workloads.

Tables XI and XII translate these findings into a practical decision framework. Table XI contains only rows whose Evidence column cites a specific measurement from this paper; Table XII contains guidance rooted in community convention and library documentation.

C. React Compiler Compatibility

An unexpected secondary finding emerges from enabling the React Compiler [13] (`babel-plugin-react-compiler` v1.0.0) across all seven conditions. Proxy-based libraries (Valtio, MobX) are *incompatible* with the current compiler: its static dependency-graph analysis cannot resolve which fields a component accesses through a runtime proxy, producing over-aggressive memoisation that *suppresses required re-renders*. Under Valtio, `DataTable` renders are incorrectly skipped in RQ3 (selection: 20→0); under MobX, *all* component renders collapse to 0 across every scenario. Both failures are silent—no compilation error or runtime warning is emitted—making the incompatibility invisible to standard production monitoring. Practitioners adopting proxy-based reactive libraries should disable the React Compiler for store-consuming components until explicit framework support is provided.

For the five compiler-compatible conditions (Redux, redux-idiomatic, Zustand, Jotai, Context), the compiler *fully eliminates* all surplus re-renders without code changes: Context RQ1 timing drops from 1 632 ms to 0.0 ms; intermediate unmemoised layers (Panel in RQ6, Layer1–3 in RQ9) are auto-memoised across all four selector-based conditions. When the compiler is enabled, the runtime distinction between Context and selector-based libraries is erased for compiler-compatible workloads. Full data are in Appendix H.

D. Threats to Validity

Internal validity. Each scenario exercises a single interaction type in isolation. The 100-row dataset does not generalise uniformly to all table sizes. **Render count** is determined solely by the library's subscription model and is invariant to row count: selector-based libraries produce 0 surplus renders on a counter increment regardless of whether 100 or 10 000 rows sit in the store. What scales with row count is the per-render reconciliation *cost*: as the DOM-visible row count grows toward the virtual-scrolling activation threshold, each surplus render becomes proportionally more expensive. At row counts where

TABLE XI
EMPIRICALLY SUPPORTED DECISION FRAMEWORK (ALL ROWS ARE DIRECTLY MEASURED IN THIS STUDY)

Usage profile	Recommended	Evidence	Note
High-frequency updates to independent slices (dashboards, real-time feeds)	Redux / Zustand / Jotai	RQ1: 0 vs. 50 re-renders	Zustand (+2 KB) delivers identical isolation at 20% of Redux's bundle cost (+10 KB)
Bundle-constrained; infrequent cross-slice updates	React Context	RQ1–RQ5, Table II	
Smallest external library overhead	Zustand	Table II: +2 KB	
Row dataset replacement (full array refresh)	All libraries equivalent	RQ5: 5 renders each	Subscribed to `rows` array reference; new reference on every refresh
Single-field partial row patch (array-reference subscription)	All libraries equivalent under `StoreAdapter`	RQ7: 10 full-table re-renders each	Uniform `StoreAdapter` interface subscribes to the `rows` array reference; every `patchRow()` replaces the array, triggering a full `DataTable` re-render in all libraries
High-frequency per-row updates; independent row subscriptions	Jotai atom families	RQ10: only the affected `RowCell` re-renders; all 99 unaffected `RowCells` and `DataTable` remain idle	One `atom<Row>` per row; each `RowCell` subscribes to its own atom; wrapper component reads only stable `rowIdsAtom`

TABLE XII
PRACTITIONER GUIDANCE (NOT DIRECTLY MEASURED IN THIS STUDY)

Consideration	Guidance	Basis
Large team; DevTools workflow	Redux Toolkit	Convention [7]
Minimal API surface	Zustand	Docs [2]
Complex derived/atomic state	Jotai	Docs [3]

virtual scrolling activates (typically >200–300 visible rows), the rendered-row count is capped by viewport capacity rather than store size, bounding each pass. The RQ1 finding (zero vs. 50 surplus re-renders) is therefore invariant to row count; absolute timing of the surplus is not. RQ7 exercises single-field patches at the array-reference level; Jotai row-level atom subscriptions are excluded by the uniform `StoreAdapter` interface. Render-count scenarios execute in fixed order (RQ1–RQ7); timing scenarios use a per-run randomised order (seeded deterministic shuffle, Section III-E), so residual ordering effects on CPU frequency scaling and JIT warm-up state are distributed across libraries and captured within the reported confidence intervals. The deterministic dataset's regular structure (five evenly distributed categories, prime-modulo value distribution) produces fixed filter-selectivity ratios under RQ2; variable selectivity (matching 2 vs. 98 of 100 rows) may produce different reconciliation profiles through the `useMemo`-guarded filter step, an interaction not explored in this study.

External validity. The benchmark uses Next.js 15 with statically pre-rendered pages and does not cover dynamic SSR, React Server Components, or Concurrent Mode features such as `useTransition`. Primary measurements use Chromium; Firefox cross-validation confirmed all render-count results are browser-engine-independent (Appendix E); WebKit/Safari cross-validation (Appendix F) confirmed the same for all seven conditions across Chromium (V8), Firefox (SpiderMonkey), and WebKit 18.4 (JavaScriptCore), addressing the engine-independence threat-to-validity for Safari. Primary timing results were collected on the Apple M5 Max 128 GB MacBook Pro primary machine; cross-hardware replication on Apple M3 Max, Intel Core i9, and Apple M1 GitHub Actions platforms confirmed identical render counts and proportional timing scaling across all three additional machines (Appendix G). The M5 Max primary values in Table VI represent lower bounds on absolute timing for mid-range hardware configurations. The two-component-tree topology was the primary scope limitation of this study: results were valid within this controlled two-component tree and may not directly transfer to production dashboards with many independent subscribing components at multiple depths. This concern is substantially mitigated: RQ6 extends the primary tree to three levels, confirming leaf isolation; RQ9 extends to five levels; and RQ11–RQ13 introduce a three-panel multi-slice topology with genuinely independent subscriptions. All results hold across all tested topologies. Production dashboards with additional complexity (deeply nested dynamic subscriptions, concurrent rendering with `useTransition`) remain future work.

Construct validity. Render count as a proxy for performance assumes that each render incurs non-negligible cost. For trivial components, an unnecessary render may be cheaper than the overhead of a selector or equality check. Per-render `actualDuration` is unavailable (Section III); aggregate render-span (Table VI) approximates the scenario-level wall-clock cost. The scenario elapsed time metric encompasses React reconciliation, browser paint, Playwright input gaps, and inter-event idle time; it is not an isolated CPU cost measure (see Section III). **Redux dashboard selector (non-idiomatic baseline).** Table IV reports `Dashboard` re-render counts for Redux under a coarse `(s) => s.app` selector, a common pre-adoption pattern that does not conform to the Redux Style Guide [7]. A fifth condition, `redux-idiomatic`, uses individual per-field `useSelector` calls with a `useMemo`-reconstructed `AppState` object in `useAppStore()`. Because every tested scenario dispatches at least one field inside `s.app`, the coarse and per-field selectors produce identical `Dashboard` render counts for all seven RQs: the isolation boundary that eliminates surplus `DataTable` re-renders is `useTableStore()`, not the selector precision of `useAppStore()`. The idiomatic variant confirms that this isolation property holds regardless of the `useAppStore` implementation style.

Equivalence testing. RQ2–RQ5 inter-library timing deltas (≤ 2.0 ms across all seven conditions) fall within all reported confidence intervals. Formal equivalence within a pre-specified δ=5 ms bound was assessed using two one-sided Welch t-tests (TOST) [10], computed by `scripts/compute_ci.py` on the steady-state timing vectors. The δ=5 ms bound was chosen because it is well below the 100 ms threshold for perceived UI responsiveness [15] and comfortably exceeds the measurement noise floor (observed CI half-widths ≤ 2.1 ms). All pairwise library comparisons for RQ2–RQ5 establish equivalence at α=0.05, confirming that the CI-overlap descriptive finding is reinforced by formal equivalence evidence within the ± 5 ms bound.[3] RQ1 is excluded from TOST because Context produces 50 renders vs. 0 for selector-based libraries, non-comparable populations for a timing equivalence claim. **Effect sizes (RQ2–RQ5).** Computed Cohen's d (pooled-SD standardised) for all $\binom{7}{2} = 21$ pairwise library comparisons per RQ spans $|d| \in [0.05, 0.60]$ for RQ2–RQ4 (mean $|d| = 0.27$; all small-to-medium). For RQ5, comparisons involving Context reach $|d| \in [0.62, 0.93]$ (conventionally "medium-large"), yet these correspond to absolute mean differences of only 0.7–0.84 ms on a platform with sub-millisecond within-library timing SD (≤ 1.4 ms). Effect-size conventions calibrated for noisy behavioural measurements do not translate directly into this precision-timing regime; within the δ=5 ms equivalence bound the practical magnitude is negligible for all comparisons. **React Compiler.** All primary results use React 19.1 with the React Compiler disabled, the default as of April 2026; compiler compatibility—including proxy-based incompatibility—is evaluated in Section V-C and Appendix H.

Reproducibility. All source code, test scripts, and raw results are available at https://github.com/KvashaIhor/state-management-benchmark [18]. The complete benchmark reproduces with `pnpm benchmark` in under two minutes; pnpm is required for lockfile-exact resolution (`pnpm-lock.yaml`). npm users may run `npm install && npm run benchmark`; the `package.json` scripts are npm-compatible, although dependency resolution may differ from the pinned `pnpm-lock.yaml` versions used in the original runs.

VI. Conclusion

I present a controlled, reproducible benchmark comparing Redux Toolkit, Zustand, Jotai, React Context, Valtio (v2.3), and MobX (v6.15) across thirteen interaction scenarios spanning bundle overhead, component re-render counts, nested-tree cascade propagation, React Compiler compatibility, and a three-panel multi-slice topology (React 19.1, React Compiler disabled unless stated, the default configuration as of April 2026). The central finding is a qualitative difference in render behaviour under irrelevant state updates: selector-based libraries (Redux, Zustand, Jotai, Valtio, MobX), when combined with `React.memo` on the child consumer component, produce zero unnecessary re-renders; React Context produces one re-render per state update regardless of which property changed. Three multi-panel scenarios (RQ11–RQ13) confirm that this isolation generalises beyond a single component tree: all six selector-based libraries produce zero cross-panel surplus renders when only one panel's slice is mutated, while Context propagates the update to all three panels in every scenario. All tested libraries produce identical render counts when the changed state is actually consumed by the component, including under partial single-field row patches (RQ7). Note that RQ7 equivalence is specific to the array-reference subscription model used in the uniform `StoreAdapter` interface; Jotai row-level atom families would likely reduce unnecessary re-renders under high-frequency per-row update workloads, a configuration excluded by the deliberate interface-parity constraint and reserved for follow-on work. A secondary finding with immediate practitioner impact: the React Compiler is *incompatible* with proxy-based libraries (Valtio, MobX), exhibiting over-memoisation that suppresses required re-renders; practitioners should disable the compiler for store-consuming components in these libraries (Appendix H). Table XI summarises the empirically grounded decision framework; Table XII separates practitioner guidance not directly measured in this study. Cross-hardware replication on Apple M3 Max, Intel Core i9, and Apple M1 CI platforms (Appendix G) confirmed that render-count results are hardware-independent, strengthening the reproducibility of the central finding.

Future work should extend the benchmark to TanStack Store; replicate the compiler pass on alternative hardware (Appendix H covers all seven conditions on Chromium/V8; cross-hardware

[3]The TOST was applied to $\binom{7}{2} \times 4 = 84$ pairwise comparisons across four RQs (seven conditions: Redux, redux-idiomatic, Jotai, Zustand, Context, Valtio, and MobX). Under Bonferroni correction ($\alpha^* = 0.05/84 \approx 0.000595$), all 84 pairs continue to establish equivalence: each CI half-width ≤ 2.1 ms falls well within the δ=5 ms bound, providing a margin of > 2.9 ms above the corrected threshold.

replication is deferred); collect Valtio and MobX timing under full-repetition conditions (N=19 steady-state runs per Appendix G methodology, now completed in this study); test Jotai row-level atom families under per-row update workloads; add isolated reconciliation-cost microbenchmarks (e.g., scoped `performance.now()` spans bracketing only the React commit phase); add memory profiling; and evaluate SSR and React Server Component scenarios.

References

[1] M. Erikson et al., "Redux Toolkit," Redux Group, v2.5, 2024. [Online]. Available: https://redux-toolkit.js.org. [Accessed: Apr. 2026].

[2] pmndrs, "Zustand," GitHub, v5.0.3, 2024. [Online]. Available: https://github.com/pmndrs/zustand. [Accessed: Apr. 2026].

[3] D. Kato et al., "Jotai: Primitive and flexible state management for React," v2.12, 2024. [Online]. Available: https://jotai.org. [Accessed: Apr. 2026].

[4] React Core Team, "React documentation," Meta Platforms, v19.1, 2024. [Online]. Available: https://react.dev. [Accessed: Apr. 2026].

[5] React Core Team, "React v18.0," *React Blog*, Mar. 29, 2022. [Online]. Available: https://react.dev/blog/2022/03/29/react-v18. [Accessed: Apr. 2026].

[6] S. Kraus, "js-framework-benchmark: A comparison of the performance of a few popular JavaScript frameworks," GitHub, 2024. [Online]. Available: https://github.com/krausest/js-framework-benchmark. [Accessed: Apr. 2026].

[7] Redux Core Team, "Redux style guide," Redux Documentation, 2024. [Online]. Available: https://redux.js.org/style-guide/style-guide. [Accessed: Apr. 2026].

[8] D. Pronina and I. Kyrychenko, "Comparison of Redux and React Hooks methods in terms of performance," in *Proc. 6th Int. Conf. on Computational Linguistics and Intelligent Systems (COLINS 2022)*, vol. 3171, CEUR Workshop Proc., pp. 791–800, 2022. [Online]. Available: https://ceur-ws.org/Vol-3171/paper59.pdf.

[9] M. Selakovic and M. Pradel, "Performance issues and optimizations in JavaScript: an empirical study," in *Proc. 38th Int. Conf. on Software Engineering (ICSE 2016)*, ACM, May 2016, pp. 61–72. DOI: https://doi.org/10.1145/2884781.2884829.

[10] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. 2013 Int. Symp. on Memory Management (ISMM '13)*, ACM, Jun. 2013. DOI: https://doi.org/10.1145/2491894.2464160.

[11] R. Ollila, N. Mäkitalo, and T. Mikkonen, "Modern web frameworks: a comparison of rendering performance," *J. Web Eng.*, vol. 21, no. 3, pp. 789–814, 2022. DOI: https://doi.org/10.13052/jwe1540-9589.21311.

[12] R. N. V. Diniz-Junior et al., "Evaluating the performance of web rendering technologies based on JavaScript: Angular, React, and Vue," in *Proc. 2022 XLVIII Latin American Computing Conf. (CLEI)*, IEEE, Oct. 2022. DOI: https://doi.org/10.1109/CLEI56649.2022.9959901.

[13] React Core Team, "React Compiler," *React Blog*, Meta Platforms, 2024. [Online]. Available: https://react.dev/learn/react-compiler. [Accessed: Apr. 2026].

[14] React Core Team, "`<Profiler>` API reference," React Documentation, v19.1, Meta Platforms, 2024. [Online]. Available: https://react.dev/reference/react/Profiler. [Accessed: Apr. 2026].

[15] J. Nielsen, "Response times: The 3 important limits," in *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993, pp. 135–137.

[16] M. Poimandres, "Valtio: Simple proxy-based state management for React," GitHub, v2.3.1, 2024. [Online]. Available: https://github.com/pmndrs/valtio. [Accessed: Apr. 2026].

[17] M. Weststrate et al., "MobX: Simple, scalable state management," GitHub, v6.15.0, 2024. [Online]. Available: https://mobx.js.org. [Accessed: Apr. 2026].

[18] I. Kvasha, "Benchmarking React State Management Libraries: Bundle Size and Re-render Performance," Zenodo, Apr. 2026. DOI: https://doi.org/10.5281/zenodo.19462716.

Appendix A
Deterministic Data Generation

```
const CATEGORIES = ['Alpha','Beta','Gamma',
                    'Delta','Epsilon'] as const

export function generateRows(count = 100): Row[] {
  return Array.from({ length: count }, (_, i) => ({
    id: `row-${i}`,
    name: `Item ${String(i+1).padStart(3,'0')}`,
    value: (i * 7919) % 1000,
    category: CATEGORIES[i % CATEGORIES.length],
    active: i % 3 !== 0,
  }))
}
```

Appendix B
Render Tracker Implementation

```
export function RenderTracker({ id, children }) {
  const isMounted = useRef(false)
  useEffect(() => {
    getRecords().push({
      id,
      phase: isMounted.current ? 'update' : 'mount',
      timestamp: performance.now(),
    })
    isMounted.current = true
  })
  return <>{children}</>
}
```

Appendix C
Raw Bundle Size Data

All three clean-build runs per library produced byte-identical output; the representative run 1 row is shown. `library_delta_kb` is the primary comparison metric; `gzip_total_kb` is a secondary cross-check.

```
# run,library,first_load_kb,delta_kb,gzip_kb
1,redux,112,10.00,244.47
1,redux-idiomatic,112,10.00,244.66
1,jotai,107,5.00,239.97
1,zustand,104,2.00,237.97
1,context,104,2.00,237.18
```

Appendix D
Webpack Alias Path Requirement

The `next.config.mjs` alias uses the **absolute** filesystem path of `store/active.tsx` as the key:

```
config.resolve.alias[
  path.resolve('./store/active.tsx')
] = path.resolve(
  `./store/${process.env.NEXT_PUBLIC_STATE_LIBRARY}/index.
    tsx`
)
```

Next.js's `tsconfig-paths-webpack-plugin` pre-resolves TypeScript `@/` path aliases to absolute paths before webpack's module resolution stage. A key of `@/store/active` would therefore already be resolved by the time the custom alias is consulted and would never match. Using an absolute path key intercepts the import at the correct resolution stage and enables genuine tree-shaking.

TABLE XIII
DATATABLE RE-RENDER COUNTS — FIREFOX 148.0.2

Scenario	Redux	Jotai	Zustand	Context
RQ1 ×50	**0**	**0**	**0**	**50**
RQ2 ×5	5	5	5	5
RQ3 ×20	20	20	20	20
RQ4 ×10	10	10	10	10
RQ5 ×5	5	5	5	5
RQ7 ×10	10	10	10	10

TABLE XIV
RQ6 NESTED TREE COUNTS — FIREFOX 148.0.2 (50 COUNTER INCREMENTS)

Component	Redux	Jotai	Zustand	Context
Shell (root)	50	50	50	50
Panel (no memo)	50	50	50	50
DataTable (`React.memo`)	**0**	**0**	**0**	**50**

APPENDIX E
FIREFOX CROSS-VALIDATION (SPIDERMONKEY)

A single Firefox 148.0.2 (Playwright *Desktop Firefox*) run across all four original libraries (`redux`, `zustand`, `jotai`, `context`) and all seven RQs. All 28 tests passed with counts identical to the Chromium primary results (Section IV). The `redux-idiomatic`, `valtio`, and `mobx` conditions were added after Firefox validation and are covered by the WebKit cross-validation (Appendix F), which confirmed identical counts across all seven conditions on WebKit/JavaScriptCore.

APPENDIX F
WEBKIT/SAFARI CROSS-VALIDATION (JAVASCRIPTCORE)

Playwright *Desktop Safari* (WebKit 18.4, JavaScriptCore) was run across all seven conditions (`redux`, `redux-idiomatic`, `zustand`, `jotai`, `context`, `valtio`, `mobx`) and all eleven render-count tests (RQ1–RQ9, RQ10). All 77 tests passed with counts byte-identical to the Chromium primary results (Section IV). This confirms that the render-isolation mechanism is browser-engine-independent across Chromium (V8), Firefox (SpiderMonkey), and WebKit (JavaScriptCore).

Timing was not collected under WebKit (TIMING_RUNS = 1, single-run warm-up pass only); the primary N=19 timing data in Table VI were collected on Chromium.

APPENDIX G
CROSS-HARDWARE REPLICATION

The complete benchmark was executed on three additional platforms using the same `pnpm benchmark` script, identical scenario parameters, and the same test toolchain as the primary run (Section III-F). All replication machines ran Next.js 15.5.0. The M3 Max and i9 platforms used N=19 steady-state runs (TIMING_RUNS=20; run 1 excluded as warm-up, df=18); the Apple M1 GitHub Actions runner used N=9 (TIMING_RUNS=10; run 1 excluded, df=8) to stay within GitHub Actions time limits. N=3 independent clean builds were used for bundle size verification.

TABLE XV
DATATABLE RE-RENDER COUNTS — WEBKIT 18.4 (JAVASCRIPTCORE). ALL SELECTOR-BASED CONDITIONS PRODUCE ZERO FOR RQ1; CONTEXT PRODUCES 50, CONSISTENT WITH V8 AND SPIDERMONKEY RESULTS.

Scenario	Rdx	RI	Jtai	Zstd	Valt	MobX	Ctx
RQ1 ×50	**0**	**0**	**0**	**0**	**0**	**0**	**50**
RQ2 ×5	5	5	5	5	5	5	5
RQ3 ×20	20	20	20	20	20	20	20
RQ4 ×10	10	10	10	10	10	10	10
RQ5 ×5	5	5	5	5	5	5	5
RQ7 ×10	10	10	10	10	10	10	10

Rdx=Redux, RI=`redux-idiomatic`, Jtai=Jotai, Zstd=Zustand, Valt=Valtio, Ctx=Context.

TABLE XVI
RQ6 NESTED TREE — WEBKIT 18.4 (50 COUNTER INCREMENTS). LEAF ISOLATION CONFIRMED ON JAVASCRIPTCORE.

Component	Rdx	RI	Jtai	Zstd	Valt	MobX	Ctx
Shell (root)	50	50	50	50	50	50	50
Panel (no memo)	50	50	50	50	50	50	50
DataTable (leaf)	**0**	**0**	**0**	**0**	**0**	**0**	**50**

Platforms:

- Apple M3 Max 128 GB MacBook Pro (independent collaborator machine, macOS);
- Intel Core i9 MacBook Pro (macOS);
- Apple M1 GitHub Actions *macos-14* shared runner.

Render counts (Tables III–VII) were byte-identical to the primary Chromium results on all three replication platforms and are not repeated here.

Table XVII compares First Load JS bundle sizes. Bundle sizes are identical across all four platforms (Redux > Jotai > Zustand ≈ Context).

Table XVIII reports render-span timing for the Context library, which is the most informative representative: it is the only library with a non-zero DataTable render-span in RQ1 (50 re-renders), and its RQ2–RQ5 timing lies within 2–3 ms of Redux, Jotai, and Zustand on every platform. Selector-based libraries produce mean = 0.0 ms on all platforms for RQ1 (zero renders; not shown).

TABLE XVII
BUNDLE SIZES ACROSS PLATFORMS (FIRST LOAD JS, UNCOMPRESSED, KB; LIBRARY DELTA ABOVE 102 KB SHARED BASELINE IN PARENTHESES)

Library	M5 Max	M3 Max	i9	M1 CI
Redux	112 (+10)	112 (+10)	112 (+10)	112 (+10)
Jotai	107 (+5)	107 (+5)	107 (+5)	107 (+5)
Zustand	104 (+2)	104 (+2)	104 (+2)	104 (+2)
Context	104 (+2)	104 (+2)	104 (+2)	104 (+2)

TABLE XVIII
DATATABLE RENDER-SPAN TIMING ACROSS PLATFORMS — CONTEXT LIBRARY, MEAN [±HALF 95% CI] (MS). M3 MAX AND I9: N=19 STEADY-STATE RUNS (t-DIST., df=18); M1 CI: N=9 (df=8); RUN 1 EXCLUDED AS WARM-UP ON ALL PLATFORMS [10]. M5 MAX VALUES ARE REPRODUCED FROM TABLE VI FOR REFERENCE. SELECTOR-BASED LIBRARIES: 0.0 [±0.0] MS ON ALL PLATFORMS FOR RQ1 (OMITTED).

Scenario	M5 Max	M3 Max	i9	M1 CI
RQ1 ×50	1632.1 [±0.5]	1634.2 [±4.2]	1634.3 [±3.1]	6141.9 [±393.7]
RQ2 ×5	213.0 [±0.7]	219.0 [±0.8]	212.7 [±1.0]	242.1 [±10.9]
RQ3 ×20	631.9 [±0.7]	632.3 [±1.9]	633.6 [±2.9]	2414.2 [±145.4]
RQ4 ×10	298.1 [±1.8]	295.9 [±0.6]	344.9 [±11.2]†	1096.1 [±137.1]
RQ5 ×5	131.1 [±0.5]	132.3 [±1.9]	131.7 [±0.4]	523.8 [±46.8]

†IQR ≈ ±35 ms; consistent with thermal throttling on x86 under repeated sort operations. M1 CI intervals are wider due to shared-runner resource contention; treat as indicative only.

APPENDIX H
REACT COMPILER SENSITIVITY

React Compiler [13] (`babel-plugin-react-compiler` v1.0.0) was evaluated across all seven conditions with N=20 timing runs (N=19 steady-state, run 1 excluded as warm-up), following the same protocol as the primary study. The compiler is opt-in via `REACT_COMPILER=1` (injects `experimental.{reactCompiler:true}` into `next.config.mjs`); all other build parameters are unchanged.

Proxy-based library incompatibility. Two conditions — Valtio and MobX — exhibit *incompatibility* with the React Compiler. Both use runtime proxy-based reactive tracking (`useSnapshot` in Valtio; `observer()` in MobX) rather than explicit selector functions. The compiler's static dependency-graph analysis cannot resolve which state fields a component accesses through a runtime proxy, causing over-aggressive memoisation: under Valtio, DataTable renders are incorrectly suppressed in RQ3 (selection: 20→0); under MobX, *all* component renders collapse to 0 across every scenario. Both conditions violate the test contract; their timing data are excluded from Table XXI and from the analysis below. Practitioners using proxy-based libraries should disable the compiler for store-consuming components or await explicit framework support.

Bundle overhead. The compiler adds ≈ + 1–+2 KB to the primary route chunk and +1–+2 KB to First Load JS across the five compatible conditions (Context, Redux, redux-idiomatic, Zustand, Jotai).

Primary-route render counts. Table XIX compares Dashboard and DataTable render counts with and without the compiler. For the four selector-based conditions (Redux, redux-idiomatic, Zustand, Jotai), DataTable counts are *unchanged*: RQ1 stays 0, RQ2–5 retain their natural render counts, confirming that the compiler does not interfere with subscription-hook primitives. The only universal change is in RQ5: Dashboard drops 5→0 across *all* compatible conditions (compiler detects Dashboard does not read `rows`). For the Context condition, the compiler additionally eliminates DataTable's 50 surplus renders in RQ1 (50→0).

Generalised cascade elimination. Table XX extends the earlier Context finding to selector-based conditions. For selector-based libraries, Panel (RQ6) and Layer1–3 (RQ9) were not previously memoised and re-rendered N times on parent re-renders even though they do not subscribe to `counter`. The compiler auto-memoises all non-subscribing intermediate components across all four compatible selector-based conditions. DataTable at the leaf was already 0 for selector-based libraries and remains 0. For Context, DataTable at the leaf drops 50→0 in both RQ6 and RQ9 (compiler detects DataTable reads `rows` from context, which is unchanged on counter increments).

TABLE XIX
REACT COMPILER — PRIMARY ROUTE RENDER COUNTS (DASH = DASHBOARD, DT = DATATABLE). SELECTOR-BASED: REDUX SHOWN; ALL FOUR CONDITIONS IDENTICAL. CONTEXT: ONLY CONDITION WHERE DT COUNT CHANGES. BASELINE REPRODUCES TABLE III.

Scenario	Dash base	Dash +cmp	DT base	DT +cmp
Context condition				
RQ1 ctr ×50	50	50	**50**	**0**
RQ2 filt ×5	5	5	5	5
RQ3 sel ×20	20	20	20	20
RQ4 sort ×10	10	10	10	10
RQ5 rows ×5	5	**0**[a]	5	5
Selector-based (Redux representative)				
RQ1 ctr ×50	0	0	0	0
RQ2 filt ×5	5	5	5	5
RQ3 sel ×20	20	20	20	20
RQ4 sort ×10	10	10	10	10
RQ5 rows ×5	5	**0**[a]	5	5

[a]Dashboard eliminated on rows refresh because Dashboard does not read `rows`. DataTable retains its subscription renders (correct).

Mechanism. The compiler performs static dependency-graph analysis. `DataTable` in Context RQ1: its render tree reads `rows` via `useContext(StateCtx)`; the compiler verifies `counter` is not read and applies automatic memoisation. Intermediate layers (Panel, Layer1–Layer3): their JSX output depends on no state and receives no new props on parent re-render; the compiler elides the cascade for *all* compatible conditions. K-consumers in Context RQ8: subscribed fields do not change on counter increments; the compiler memoises all K instances. This static approach succeeds for Context and explicit-selector hooks (Redux, Zustand, Jotai), whose dependencies are compile-time-visible. It *fails* for proxy-based reactive libraries (Valtio, MobX), where dependencies are resolved at runtime through transparent proxy interception and are therefore opaque to the compiler.

Timing results. Table XXI reports N=19 steady-state DataTable render-span timing with the compiler enabled for the five compatible conditions. Context RQ1 timing drops from 1632.1 ms to 0.0 ms (DataTable renders 50→0), eliminating the scenario entirely. For all other scenario–condition pairs, timing is within 2 ms of the baseline (Table VI), confirming that compiler memoisation overhead is negligible. One Context run produced timing anomalies in RQ2–5 (likely a GC pause); median values (211, 632, 299, 132 ms) are consistent with baseline and are shown in Table XXI.

Implication. For Context and the four explicit-selector

TABLE XX
REACT COMPILER — NESTED AND FAN-OUT SCENARIOS. CONTEXT: BASELINE FROM TABLES VII, VIII, AND X. SELECTOR-BASED: REDUX REPRESENTATIVE; REDUX-IDIOMATIC, ZUSTAND, AND JOTAI IDENTICAL. ROOT SUBSCRIBERS (SHELL/DEEPSHELL) RETAIN THEIR RE-RENDERS AS THEY GENUINELY READ THE CHANGED STATE.

Component / Scenario	Baseline	+Compiler
Context condition		
RQ6 (3-level tree, 50 increments)		
Shell (root)	50	50
Panel (no memo)	50	**0**
DataTable (leaf)	**50**	**0**
RQ9 (5-level tree, 50 increments)		
DeepShell (root)	50	50
Layer1 (no memo)	50	**0**
Layer2 (no memo)	50	**0**
Layer3 (no memo)	50	**0**
DataTable (leaf)	**50**	**0**
RQ8 (K-consumer fan-out, 50 increments)		
K=3 total surplus	150	**0**
K=10 total surplus	500	**0**
Selector-based conditions (Redux representative)		
RQ6 (3-level tree, 50 increments)		
Shell (root)	50	50
Panel (no memo)	**50**	**0**
DataTable (leaf)	0	0
RQ9 (5-level tree, 50 increments)		
DeepShell (root)	50	50
Layer1 (no memo)	**50**	**0**
Layer2 (no memo)	**50**	**0**
Layer3 (no memo)	**50**	**0**
DataTable (leaf)	0	0

TABLE XXI
REACT COMPILER ENABLED — DATATABLE RENDER-SPAN (MS), N=19 STEADY-STATE MEAN [±HALF-95% CI]. CONTEXT USES MEDIAN (ONE OUTLIER RUN). VALTIO AND MOBX EXCLUDED (PROXY-BASED INCOMPATIBILITY). ALL SELECTOR-BASED RQ1 = 0.0 MS (ZERO RENDERS). COMPARE WITH TABLE VI FOR BASELINE VALUES.

Scenario	Redux	Rdx-Idm	Zustand	Jotai	Context
RQ1	0.0	0.0	0.0	0.0	0.0
RQ2	212.1 [±0.9]	213.3 [±1.5]	211.8 [±0.8]	210.8 [±1.0]	211*
RQ3	631.3 [±0.5]	631.5 [±0.4]	632.5 [±1.8]	631.9 [±0.5]	632*
RQ4	297.2 [±0.6]	297.7 [±0.5]	297.9 [±0.4]	297.7 [±0.6]	299*
RQ5	131.4 [±0.5]	131.3 [±0.5]	131.5 [±0.5]	131.6 [±0.5]	132*

*Context median (one of 19 runs triggered a GC-related anomaly; 18 clean-run median consistent with baseline).

conditions, the React Compiler is a transparent performance enhancement: it eliminates surplus re-renders without code changes, adds at most ≈2 KB per route, and Context RQ1 timing collapses from 1632 ms to 0.0 ms. The compiler also generalises cascade elimination (Panel, Layer1–3) to selector-based libraries, a finding not visible in a Context-only sensitivity analysis. However, proxy-based reactive libraries (Valtio, MobX) are *incompatible* with the current compiler version; practitioners should disable the compiler for those store-consuming components or await explicit framework support. Cross-hardware replication of the compiler pass is deferred to follow-on work.